On the Expressivity of GNNs for Boolean Satisfiability

Saku Peltonen, Roger Wattenhofer
ETH Zurich

Introduction

Boolean satisfiability (SAT) is NP-hard, yet many real instances can be solved efficiently in practice. Classical solvers achieve this with carefully designed heuristics, but recent work asks whether such solver behavior can be learned rather than engineered by hand. This motivates learning-based SAT solvers, where Graph Neural Networks (GNNs) are a natural choice because Boolean formulas admit natural graph representations.

We study the expressive power of GNNs for SAT through the lens of the Weisfeiler-Leman (WL) hierarchy. Our main result shows that, in general, even the full WL hierarchy cannot distinguish satisfiable from unsatisfiable instances. We further show that higher-order WL indistinguishability leads to concrete limitations for WL-bounded solvers that set variables sequentially, and we analyze the expressivity needed for regular, random, and planar SAT instances.

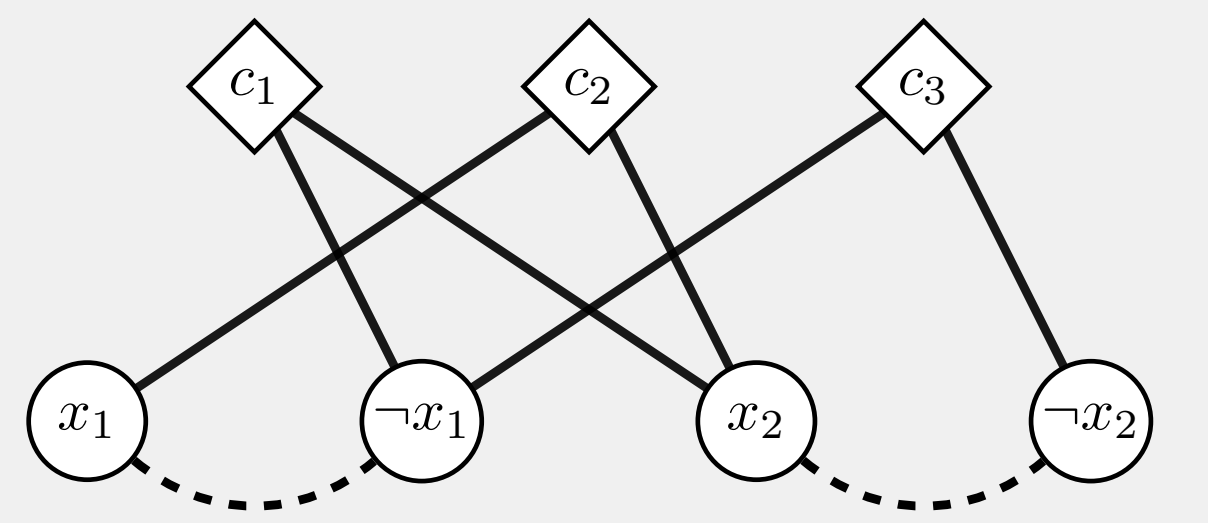
Empirically, we evaluate random instances from G4SAT and industrial benchmarks from the International SAT Competition. Random instances are often distinguishable, whereas industrial instances frequently require substantially more expressive models to predict satisfying assignments.

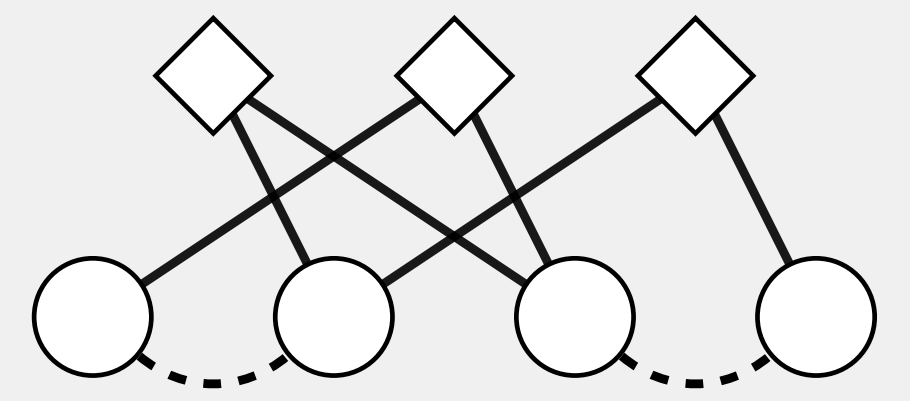
SAT is a highly structured reasoning problem, making it a natural stress test for GNN expressivity.

Graph Representation

Literal-Clause Graph (LCG)

$f = (\neg x_1 \vee x_2) \wedge (x_1 \vee x_2) \wedge (\neg x_1 \vee \neg x_2)$

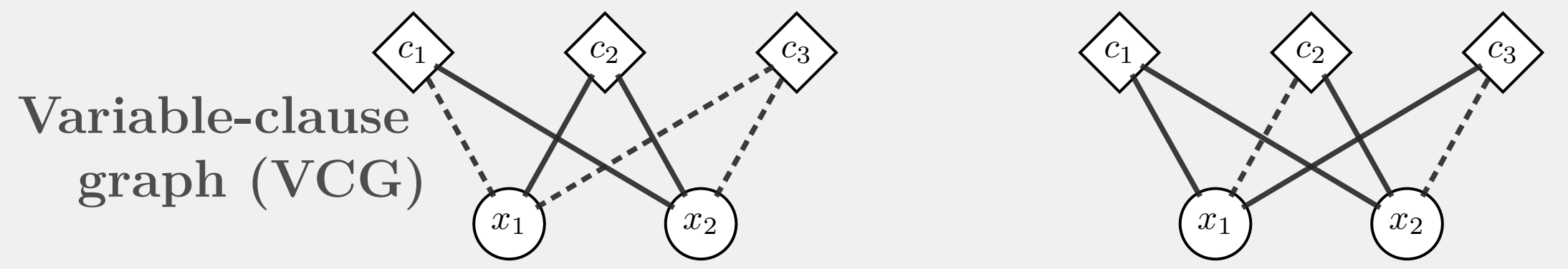




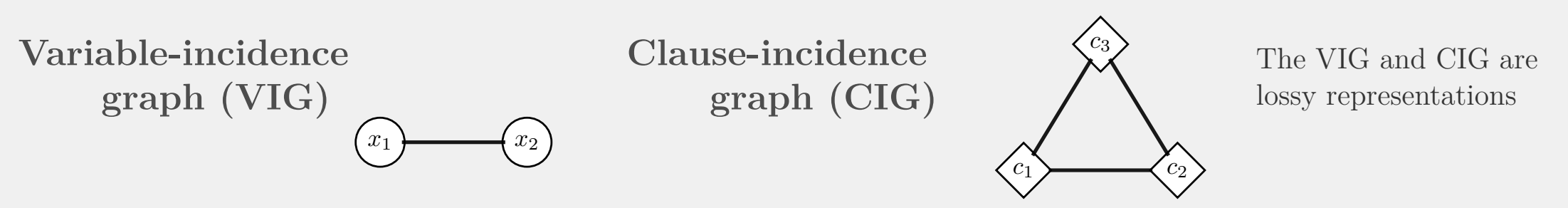
The GNN does not see node labels.

Side note: other graph representations

Left: $f = (\neg x_1 \vee x_2) \wedge (x_1 \vee x_2) \wedge (\neg x_1 \vee \neg x_2)$
Right: $f' = (\mathbf{x_1} \vee x_2) \wedge (\neg \mathbf{x_1} \vee x_2) \wedge (\mathbf{x_1} \vee \neg x_2)$



The VCG is not invariant to sign flips, whereas LCG is. f and f' represent the same formula up to flipping a literal, but their graphs are different.


Variable-incidence graph (VIG)
x_1
x_2
Clause-incidence graph (CIG)
c_3
c_1
c_2
The VIG and CIG are lossy representations

Background: CNF Formulas

A *literal* is a variable x or its negation $\neg x$. A *clause* is a disjunction of literals, such as $(x_1 \vee \neg x_3 \vee x_7)$. A SAT instance in *conjunctive normal form* (CNF) is a conjunction of clauses:

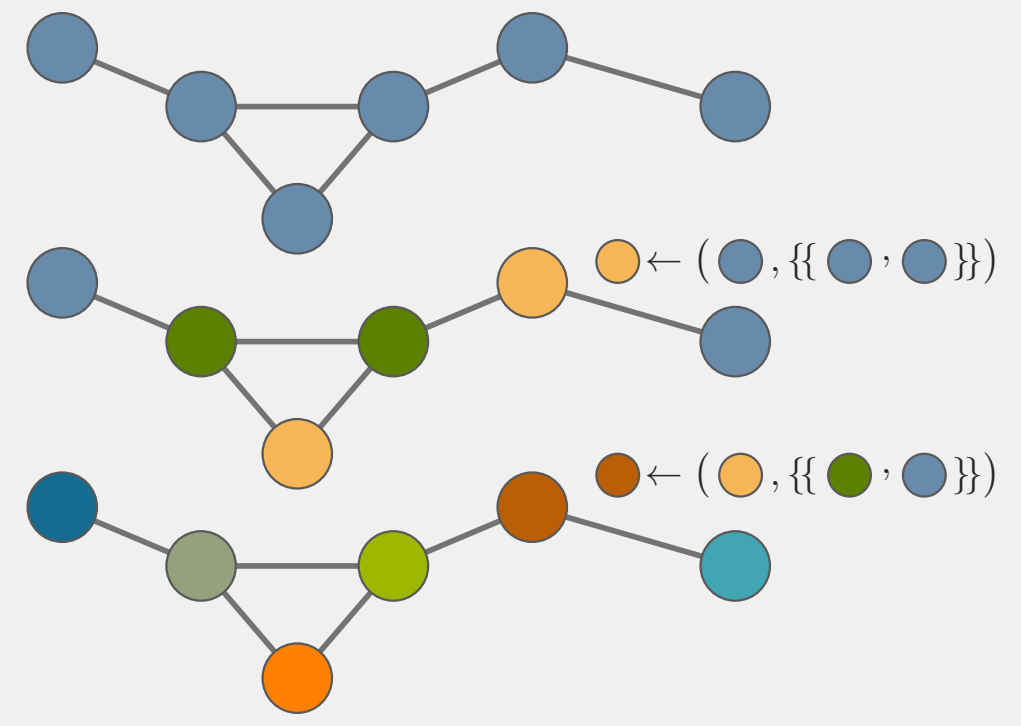
$$f = c_1 \wedge c_2 \wedge \cdots \wedge c_m.$$

The formula is satisfiable if there exists an assignment to the variables that makes every clause true.

Background: Message Passing

Message Passing Neural Networks (MPNNs) update node states in synchronous rounds. With initial features $s_v^0 \in \mathbb{R}^d$, each round computes

$$a_v^\ell = \mathrm{agg}(\{\{s_w^{\ell-1} : w \in N(v)\}\}), \qquad s_v^\ell = \mathrm{upd}(s_v^{\ell-1}, a_v^\ell).$$



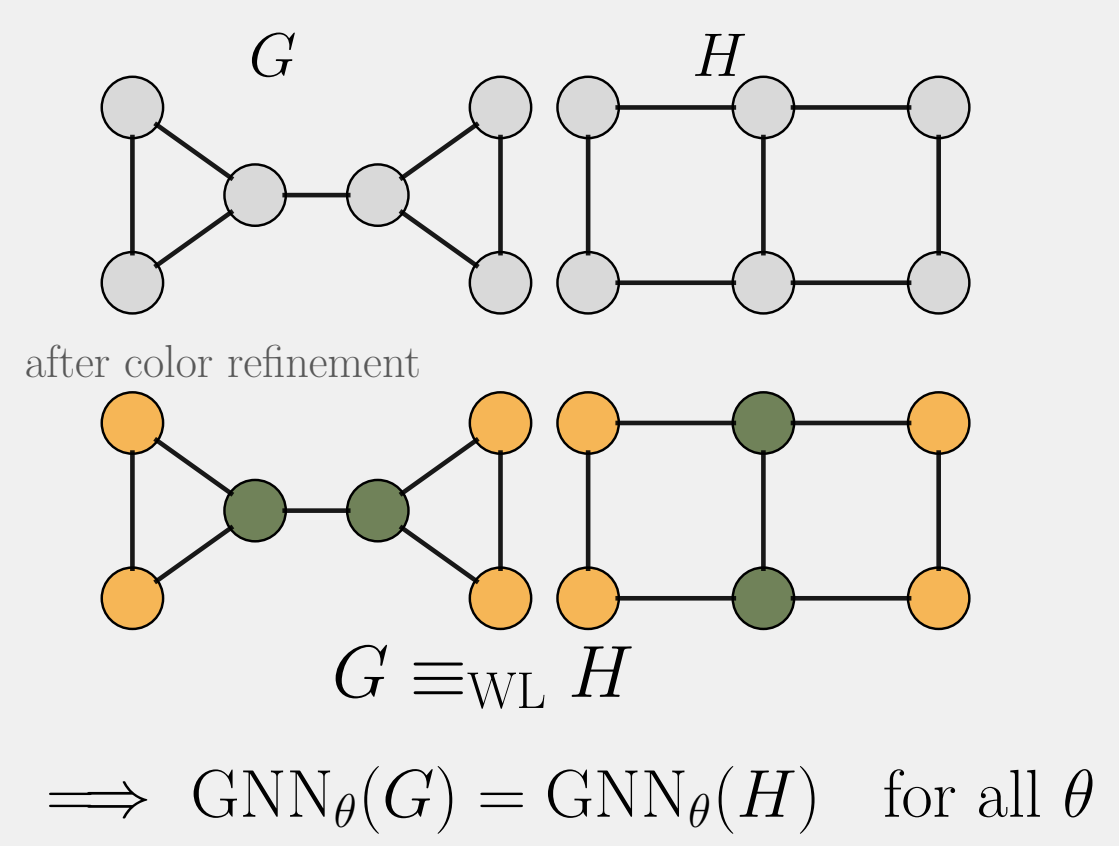
Color refinement

Starting from colors χ^0 (constant if not given), color refinement replaces learnable aggregation and updates states by a hash-like relabeling:

$$\chi_v^\ell = \left(\chi_v^{\ell-1}, \{\{\chi_w^{\ell-1} : w \in N(v)\}\}\right).$$

The procedure repeats until the coloring stabilizes.

Weisfeiler-Leman (WL) test



$$G \equiv_{\mathrm{WL}} H$$
$$\implies \mathrm{GNN}_\theta(G) = \mathrm{GNN}_\theta(H) \quad \text{for all } \theta$$

The WL test is a graph isomorphism heuristic. It performs color refinement on the disjoint union $G \sqcup H$.

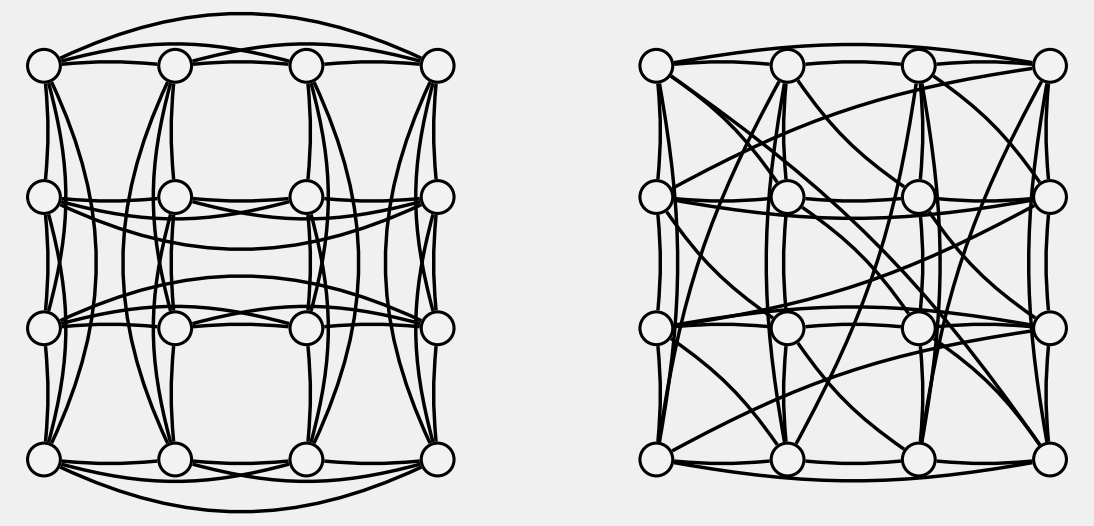
We say that WL distinguishes G and H if the final stable coloring gives different multisets of node colors for the graphs.

The expressive power of message-passing GNNs is bounded by the WL-test.

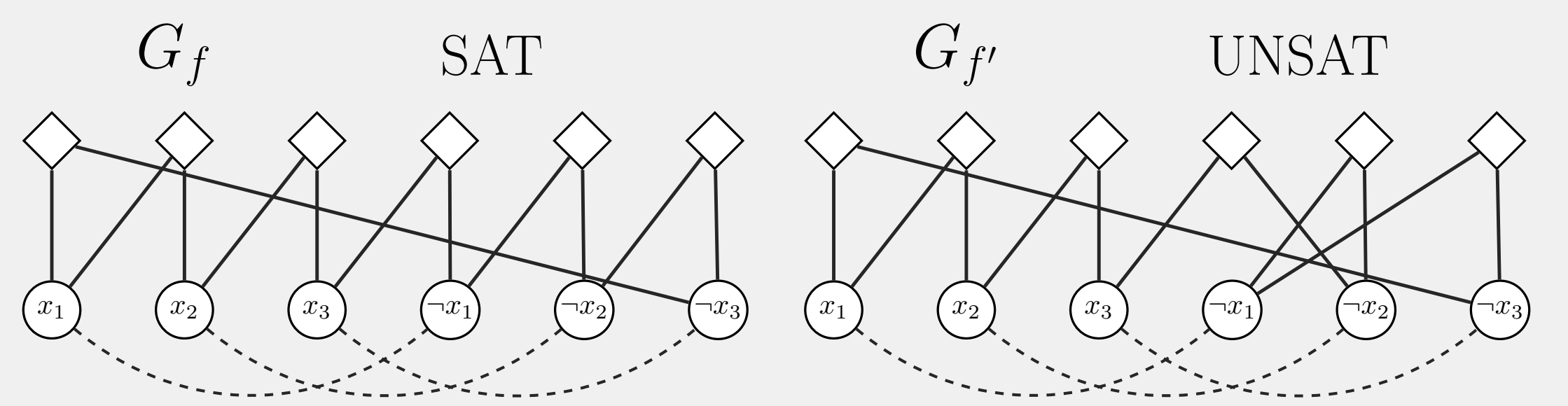
k-WL test

The k-WL hierarchy consists of increasingly powerful tests that distinguish between increasingly similar graphs. Instead of nodes, these tests operate on k-tuples of nodes. Cai, Fürer, and Immerman [1] proved that there are graph pairs on $O(n)$ nodes that remain indistinguishable to n-WL.

Example graph pair not distinguished by 3-WL:



Example Where GNNs Fail



Left: $f = (x_1 \vee \neg x_3) \wedge (x_1 \vee x_2) \wedge (x_2 \vee x_3) \wedge (x_3 \vee \neg x_1) \wedge (\neg x_1 \vee \neg x_2) \wedge (\neg x_2 \vee \neg x_3)$
Right: $f' = (x_1 \vee \neg x_3) \wedge (x_1 \vee x_2) \wedge (x_2 \vee x_3) \wedge (x_3 \vee \neg \mathbf{x_2}) \wedge (\neg x_1 \vee \neg x_2) \wedge (\neg \mathbf{x_1} \vee \neg x_3)$

$$G_f \equiv_{\mathrm{WL}} G_{f'}$$
$$\implies \mathrm{GNN}_\theta(G_f) = \mathrm{GNN}_\theta(G_{f'}) \quad \text{for all } \theta$$

This example can be generalized to an NP-complete family.

Theorem. 3-regular SAT * is NP-complete, but instances with the same number of variables are indistinguishable by WL.

*All variables and their negations are exactly in 3 clauses, and all clauses contain exactly 3 literals.

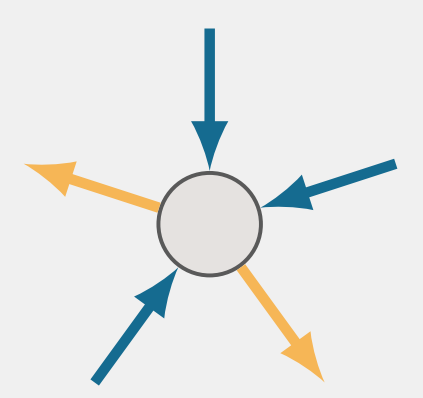
n-WL Indistinguishable Instances

Theorem. There are n-WL indistinguishable formulas f, f' with $O(n)$ variables and $O(n)$ clauses such that f is SAT and f' is UNSAT.

Construction

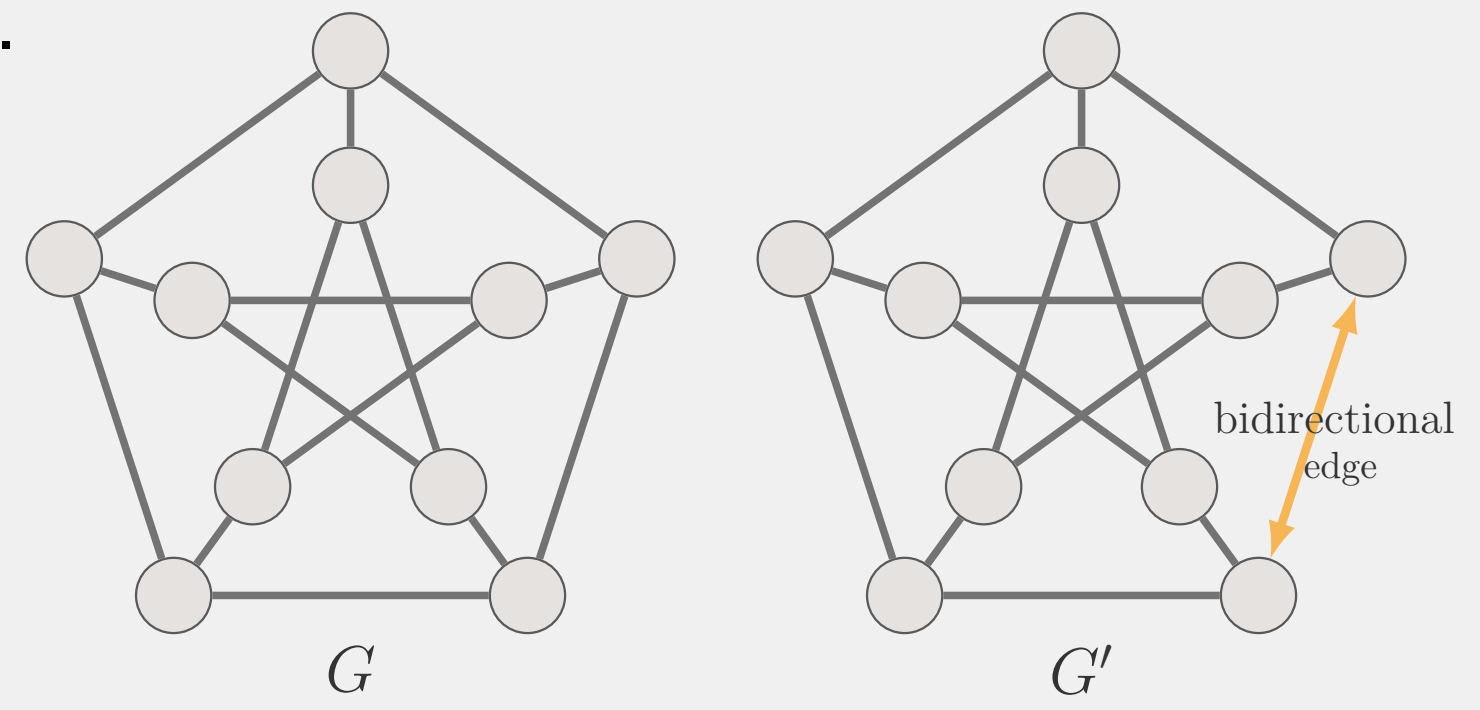
We encode the existence of an *even orientation*, where every vertex has even out-degree:

$$\forall v \in V : \; d^{\mathrm{out}}(v) = 0 \pmod 2.$$



It can be shown that an even orientation exists on a connected graph iff $|E|$ is even. Let f encode that G has an even orientation. We choose G to be a 3-regular expander graph and construct a companion graph G' with one bidirectional edge.

f SAT $\iff$ f' UNSAT

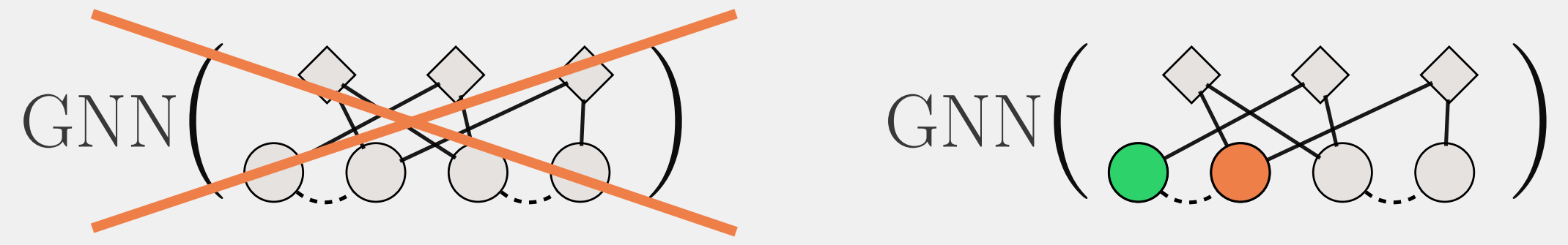


We show that $G_f \equiv_{n-WL} G_{f'}$.

The WL hierarchy cannot in general distinguish SAT instances from UNSAT.

Implication for Sequential Solvers

Predicting whether a formula is SAT straight away is arguably hard. In practice, GNN SAT solvers like QuerySAT [5] set variables sequentially. Does this overcome the expressivity limitations?



Theorem. Even with $\Omega(n)$ variables assigned, a WL-powerful GNN cannot always distinguish between a SAT and an UNSAT instance.

Positive Results

Expressivity and complexity are distinct: NP-hardness does not imply WL indistinguishability.

PlanarSAT

There are also interesting families of SAT formulas where GNNs can distinguish instances. For example, PlanarSAT is NP-complete, but all instances can be distinguished by 4-WL (follows from [3]).

Theorem. PlanarSAT instances are fully distinguishable by 4-WL.

Random SAT instances

Random SAT is often easier from an expressivity perspective. We prove this for a specific random generation method and validate the phenomenon experimentally on a broader family of random instances.

Experiments

Setup. Does a WL-powerful GNN have the *necessary* expressivity to output a satisfying assignment? We study this for a satisfiable CNF formula f with WL-equivalence classes of literals $L_1, \ldots, L_s$. We augment f with equality constraints inside each WL-equivalence class:

$$f_{\mathrm{WL}} = f \wedge \bigwedge_{i=1}^{s} \mathrm{AllEqual}(L_i).$$

$r_{\mathrm{crit}} \in \{0, \ldots, n-1\} \cup \{\infty\}$ smallest iteration such that f_r is satisfiable,
$r_{\mathrm{conv}} \in \{0, \ldots, n-1\}$ number of iterations until convergence.

If $r_{\mathrm{crit}} = \infty$, solving f requires expressivity beyond WL.

Results

Industrial benchmarks from the international SAT Competition [2] often require more expressive models to predict a satisfying assignment, whereas random instances from a GNN SAT benchmark (G4SATBench [4]) are easier from the expressivity perspective.

GNN benchmarks and training relies on random SAT instances, but these are often easier from the expressivity point of view.

family	r_{crit}	r_{conv}	Variables	Count
argumentation	2.94 ± 0.44	4.31 ± 0.87	1266 ± 625	16
brent-equations	31.11 ± 3.86	38.78 ± 12.34	72398 ± 9398	9
	unsat	59.00 ± 0.00	78934 ± 5098	2
circuit-multiplier	unsat	7.18 ± 0.40	1075 ± 50	11
coloring	unsat	6.71 ± 4.61	15563 ± 28396	14
cryptography	15.74 ± 14.67	17.63 ± 14.34	41510 ± 29705	19
	unsat	18.74 ± 13.38	19257 ± 37634	23
hamiltonian	4.17 ± 0.51	5.44 ± 0.51	550 ± 45	18
heule-folkman	unsat	4.91 ± 0.30	16614 ± 1103	11
heule-nol	unsat	8.60 ± 1.26	1419 ± 0	10
maxsat-optimum	26.64 ± 3.64	29.27 ± 2.49	22157 ± 5623	11
	unsat	59.00 ± 0.00	27597 ± 8713	7
scheduling	12.14 ± 2.19	16.14 ± 3.48	16459 ± 3223	7
	unsat	30.86 ± 13.37	35633 ± 29001	28

family	difficulty (size)	r_{crit}	r_{conv}	Variables	Count
3-sat	easy	2.97 ± 0.18	3.68 ± 0.47	26 ± 9	1000
	medium	3.00 ± 0.04	3.92 ± 0.28	119 ± 47	1000
	hard	3.00 ± 0.00	4.00 ± 0.00	250 ± 29	100
	hard+	3.00 ± 0.00	4.00 ± 0.00	921 ± 48	24
	hard++	3.08 ± 0.28	4.00 ± 0.00	5001 ± 62	25
k-domset	easy	4.11 ± 0.71	5.61 ± 0.93	39 ± 12	1000
	medium	4.33 ± 0.58	5.50 ± 0.72	88 ± 18	1000
	hard	4.42 ± 0.67	5.54 ± 0.69	137 ± 22	100
k-vercov	easy	4.75 ± 1.14	6.18 ± 1.38	39 ± 13	993
		unsat	4.00 ± 0.00	26 ± 8	7
	medium	4.94 ± 1.00	5.88 ± 1.08	96 ± 20	1000
	hard	5.00 ± 1.01	5.80 ± 0.85	179 ± 25	100
sr	easy	2.00 ± 0.05	3.00 ± 0.06	25 ± 9	1000
	medium	2.01 ± 0.12	3.00 ± 0.00	118 ± 47	1000
	hard	2.05 ± 0.22	3.00 ± 0.00	299 ± 62	100

References

[1] Jin-Yi Cai, Martin Fürer, and Neil Immerman. An optimal lower bound on the number of variables for graph identification. *Combinatorica*, 12(4):389–410, 1992.

[2] Marijn JH Heule, Markus Iser, Matti Järvisalo, and Martin Suda. Proceedings of SAT competition 2024: Solver, benchmark and proof checker descriptions. 2024.

[3] Sandra Kiefer, Ilia Ponomarenko, and Pascal Schweitzer. The Weisfeiler–Leman dimension of planar graphs is at most 3. *J. ACM*, 66(6), November 2019.

[4] Zhaoyu Li, Jinpei Guo, and Xujie Si. G4SATBench: Benchmarking and advancing SAT solving with graph neural networks. *Transactions on Machine Learning Research*, 2024.

[5] Emils Ozolins, Karlis Freivalds, Andis Draguns, Eliza Gaile, Ronalds Zakovskis, and Sergejs Kozlovics. Goal-aware neural SAT solver. In *2022 International Joint Conference on Neural Networks (IJCNN)*, pages 1–8. IEEE, July 2022.